

ゼロから Python で行列簡約化

電気通信大学 I 類

tenpura

2026 年 9 月 13 日

1 はじめに

B1 で MMA 平部員の Tenpura と申します。この記事では python で行列の簡約化を実装するまでの道のりを書いていきます。やろうと思った動機は暗記支援アプリ Anki のアドオンを作りたくてアドオン作成マニュアル¹⁾に従ったらよく分からない操作を延々させられて怖気づいて基礎から学びたくなったからです。あと線形代数で追試くらったからです。雪辱を果たしに。(単位はくれました)

2 実装

では作ってきたものを一つ一つ説明します。ラバーダック・デバッグって言いますよね。コードの表示に M1 の水素さんの L^AT_EX テンプレートを使いました。非常に便利なのでここ²⁾から使いましょう。

コード 1: おまじない

```
1 import re
2 from decimal import Decimal
3 from fractions import Fraction
```

いわゆるおまじないです。競技プログラミングなどで全く意味はわからないけどなんか書かされるやつ。一応意味はググったら出ます。

- import re

Python で正規表現を扱うためのパッケージです。例えば後述の改行を表す `\n` などで動いています。

- from decimal import Decimal

少数を精度よく扱うためのパッケージです。これが無い時は少数を計算する際に `float` と呼ば

1) <https://addon-docs.ankiweb.net/>

2) <https://gitlab.mma.club.uec.ac.jp/hydrogen/latex-template>

れる高速な計算方式を使いますが、 $0.1 + 0.2 = 0.30000000000000004$ のように、非常に小さい誤差を出します。これを無くするために低速だけど正確な計算方式 decimal を使います。

- from fractions import Fraction

分数を扱うためのパッケージです。今回は割り算をしまくるので必須。電卓を使った日を覚えていますか？四則演算は瞬時にできますが割り算はきれいに表示できないやつ。これを使えば、あの 0.333333 とかの循環小数を $1/3$ と綺麗に表示してくれる計算機が作れます！感動。

コード 2: 行列入力

```
1 class Input_matrix:
2
3     def tableinput(self,a):
4         name = a
5         row = 0
6         i = 0
7         pre = []
8         print(f"行列 {name} を入力\n入力例:16 1.2 2/3 \nendと入力して行入力終了")
9
10        while row != "end":
11            row = input(str(i+1)+ "行目:")
12            if row == "end":
13                break
14            pre.insert(i, row)
15            row_count = len(pre[0].split())
16
17            if re.search(r"^[^d./ -]", row) and row != "end":
18                print("数字と-./以外入れるなポケ")
19                del pre[i]
20                continue
21
22            elif row_count != len(pre[int(i)].split()):
23                print("列数がちげーよポケ")
24                del pre[i]
25                continue
26
27            print(*pre, sep="\n")
28            i = i + 1
29
30        a = [[Fraction(x) for x in s.split()] for s in pre]
31        print(f"\n行列{name}の入力終了")
32        print(*a, sep="\n")
33        return a
```

後のプログラムで計算させる行列を入力させます。入力方式は整数, 少数, 分数を空白区切りで入力する方式です。行の入力が終わったら enter を押して, すべての入力が終わったら end と入力します。

まず

コード 3: 行列計算

```
1
2
3 class Tablecalculate:
4     def __init__(self):
5         self.A = [[0]]
6         self.B = [[0]]
7         self.C = [[0]]
8         self.n = 0
9
10    def simplify(self,a):
11        i = 0
12        j = 0
13        k = 0
14        process_row = 0
15        stop = False
16
17        gyoretsushiki = False
18        m = 1
19
20        while process_row < len(a):
21            i = process_row
22            j= 0
23            #Step1
24            while a[i][j] == 0:
25                if i == len(a)-1 and a[i][j] == 0:
26                    i = process_row
27                    j = j + 1
28                    if j > len(a[0]) - 1:
29                        stop = True
30                        break
31                    continue
32                if stop == True:
33                    break
34                i = i + 1
35            if j > len(a[0])-1:
36                break
```

```

37         print(f"a{i+1,j+1}に注目")
38         tmp_i = i
39         tmp_j = j
40     #Step2
41     k = process_row
42     while a[k][tmp_j] == 0:
43         k = k + 1
44     if process_row+1 != tmp_i+1:
45         for j in range(len(a[0])):
46             tmp = a[process_row][j]
47             a[process_row][j] = a[tmp_i][j]
48             a[tmp_i][j] = tmp
49         print(f"{process_row+1}行目 ⇔ {tmp_i+1}行目")
50         if gyoretsushiki == True:
51             m = -m
52             print(f"倍数m:{m}")
53         tmp_i = process_row
54         self.output(a)
55     #Step3
56     if a[tmp_i][tmp_j] != 1:
57         k = a[tmp_i][tmp_j]
58         print(f"{tmp_i + 1}行目 / {k}")
59         if gyoretsushiki == True:
60             m = m * k
61             print(f"倍数m:{m}")
62         for j in range(len(a[0])):
63             a[tmp_i][j] = Fraction(a[tmp_i][j], k)
64
65     self.output(a)
66     #Step4
67     for i in range(process_row if gyoretsushiki == True else 0, len(a)):
68         if a[i][tmp_j] != 0 and i != tmp_i:
69             k = a[i][tmp_j]
70             for j in range(len(a[0])):
71                 a[i][j] = a[i][j] - a[tmp_i][j] * k
72             print(f"{i + 1}行目 - {tmp_i + 1}行目 × {k}")
73
74     self.output(a)
75     process_row = process_row + 1
76     print("row=", process_row)
77     if gyoretsushiki == True:
78         print(f"倍数m:{m}")

```

```

79         return a
80
81     def output(self,a):
82         b = [[0 for j in range(len(a[0]))] for i in range(len(a))]
83         for j in range(len(a[0])):
84             for i in range(len(a)):
85                 b[i][j] = str(a[i][j])
86         print(" ", *b, sep="\n")
87
88
89 InputMatrix = Input_matrix()
90
91 A = InputMatrix.tableinput("A")
92 TableCalculate = Tablecalculate()
93 result = TableCalculate.simplify(A)
94 print("計算結果:")
95 print(*result, sep="\n")

```
